

the source of truth, Redis is a read cache only, and Kafka carries asynchronous events. Critical modules: Inventory, Payment, Order.

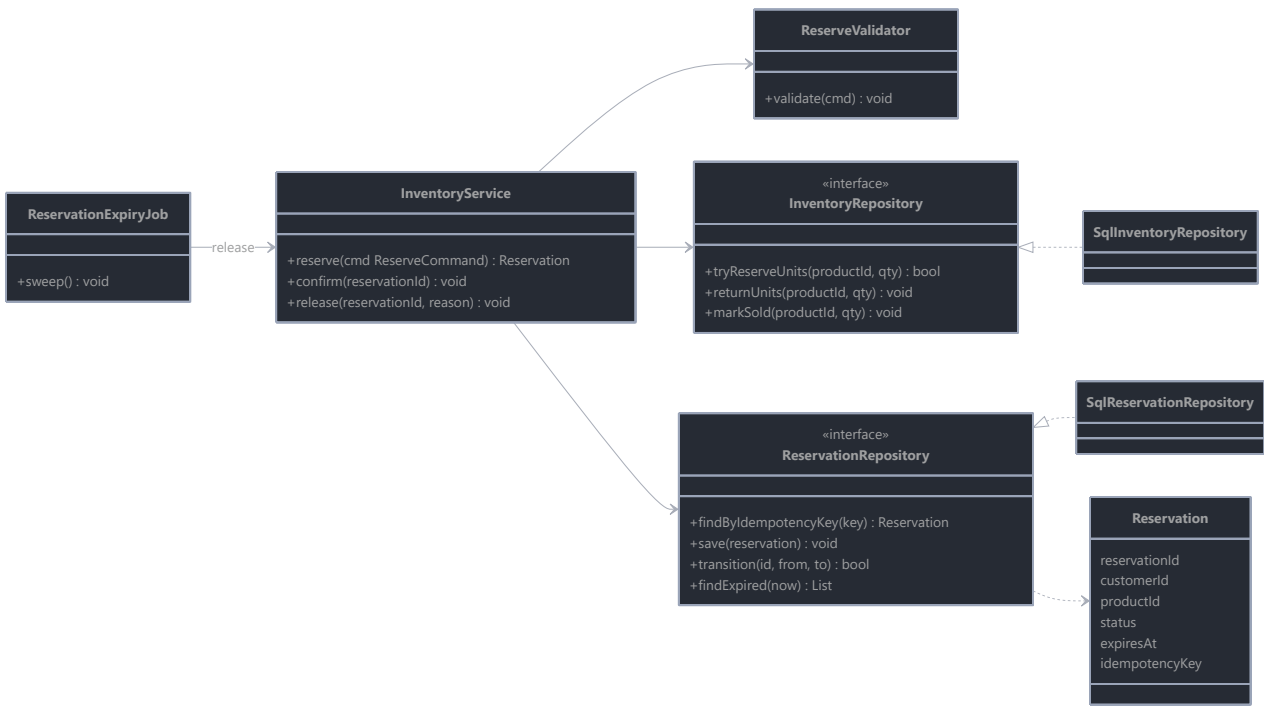
1. Service boundaries

Service	Owns	Does not own	Interfaces
Checkout (facade)	Purchase orchestration, compensation	Stock, money, order state	Calls Inventory, Order, Payment
Inventory	Stock counts, reservations, expiry	Pricing, payment	reserve, confirm, release
Payment	Payment records, provider calls, reconciliation	Stock, order state	pay, webhook. Emits PaymentCaptured, PaymentFailed
Order	Order lifecycle and state machine	Stock, payment	create, cancel. Consumes payment events. Emits OrderConfirmed, OrderCancelled
Fulfilment, Notification	Shipment, customer messages	Everything above	Consume OrderConfirmed

Each service owns its own tables. There are no cross-service joins, only API calls and events.

2. Class diagrams

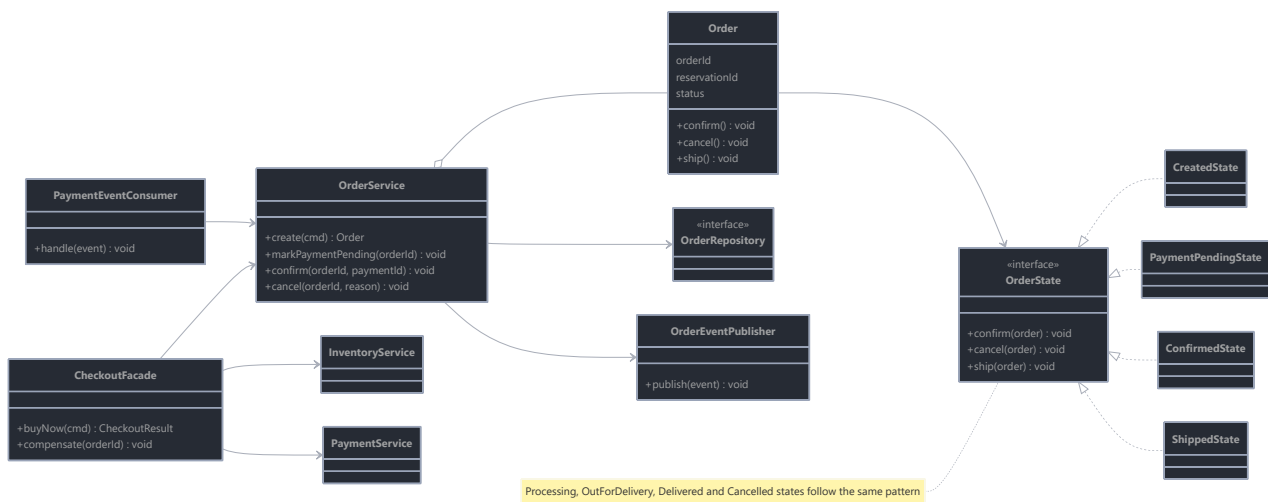
2.1 Inventory module



2.2 Payment module

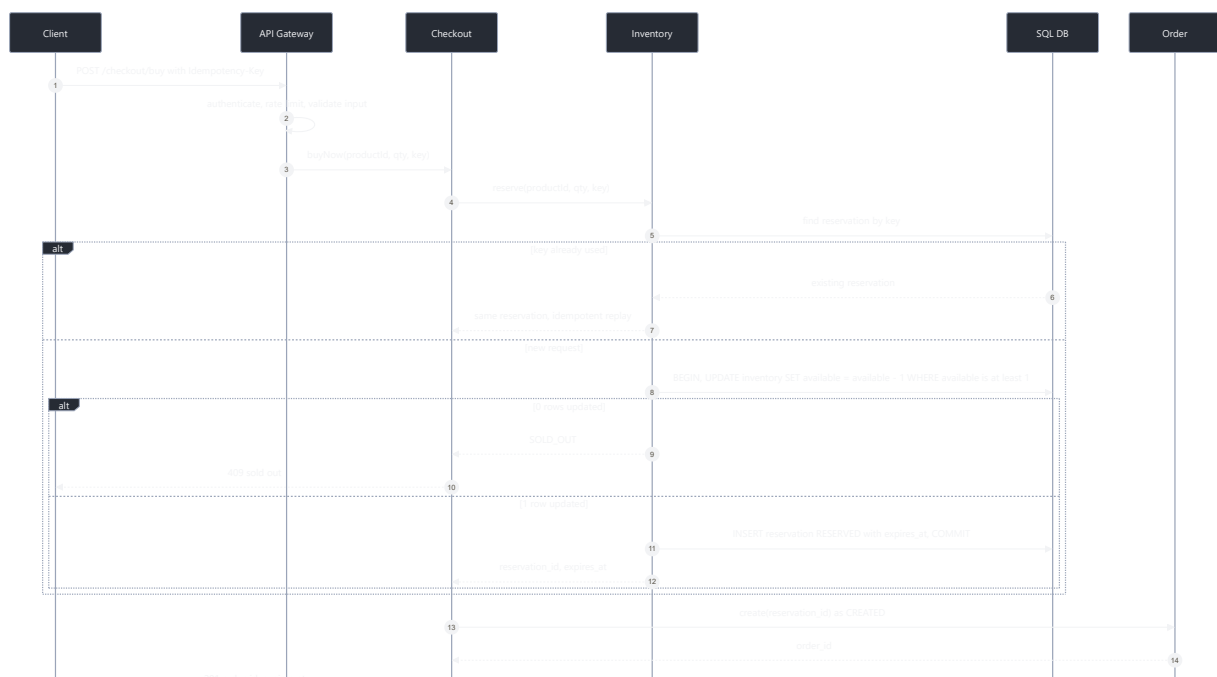


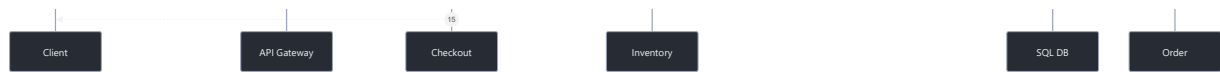
2.3 Order module and checkout facade



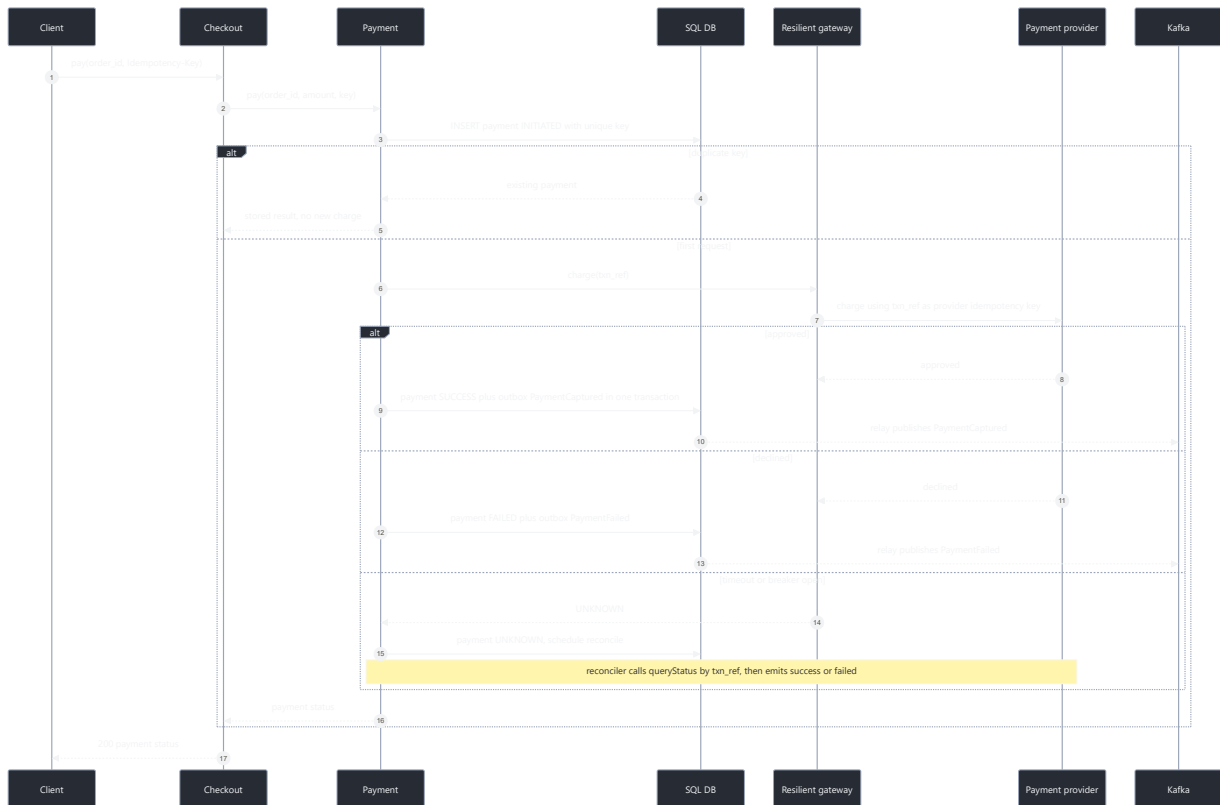
3. Sequence diagrams

3.1 Purchase and reservation

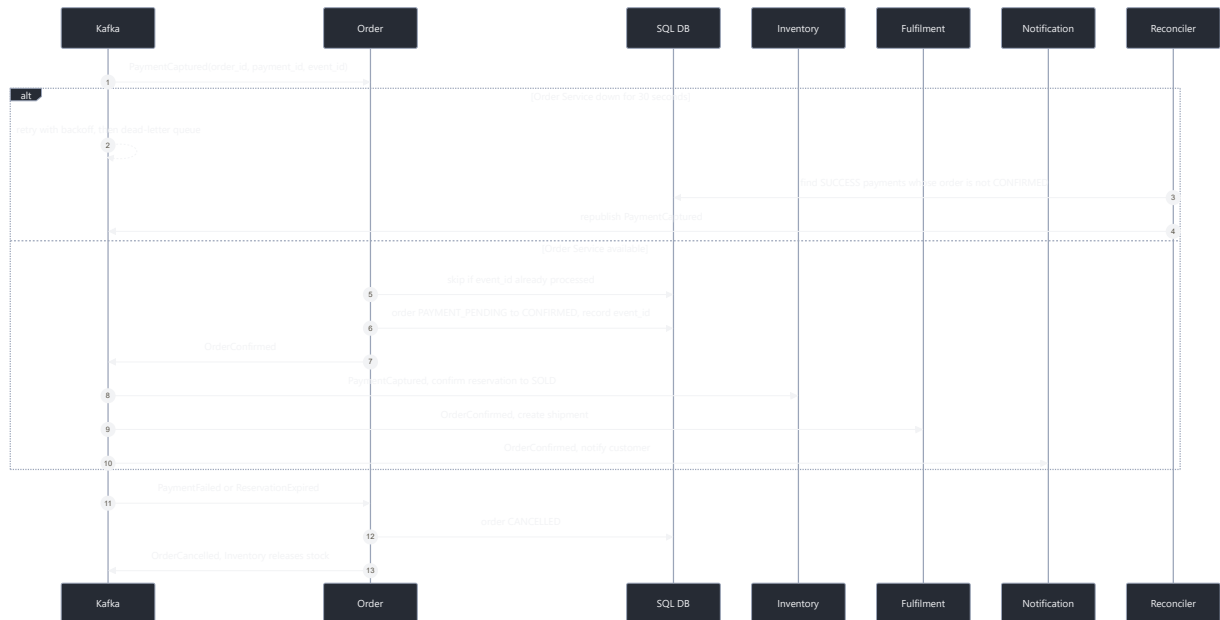




3.2 Payment

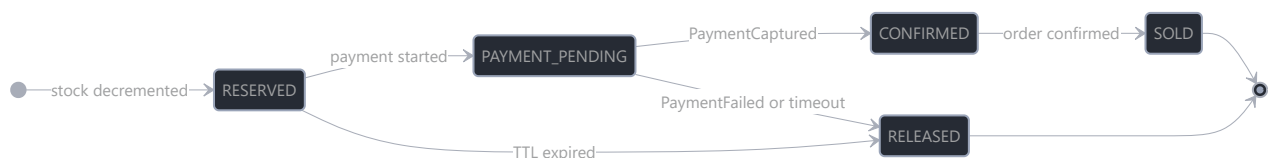


3.3 Order confirmation and recovery



4. State diagrams

4.1 Reservation lifecycle



4.2 Order lifecycle



5. Where validation, idempotency and error handling belong

Concern	Where it lives	Detail
Input validation	API Gateway, then domain validators	Gateway: auth, schema, rate limit. ReserveValidator: quantity at least 1, per-customer limit, sale is active
Idempotency	Database constraints in each service	Reservation: unique <code>idempotency_key</code> . Payment: unique key, plus <code>txn_ref</code> sent to the provider. Order: unique <code>reservation_id</code> . Consumers: <code>processed_events(event_id)</code>
Concurrency	Inventory repository	One conditional UPDATE in a short transaction. Status changes use guarded updates: <code>WHERE status IN (...)</code> , so repeated release is a no-op
Client errors	Checkout facade	400 invalid input, 409 sold out or duplicate in progress, 503 retryable dependency failure
Payment failures	ResilientGateway	Timeout, retry with backoff, circuit breaker. A timeout becomes UNKNOWN, never FAILED, until reconciled
Async failures	Kafka consumers	Backoff, then dead-letter queue. Reconciler republishes missed events
Compensation	Event handlers	PaymentFailed: release stock and cancel order. Payment captured after the reservation was released: refund. The expiry sweeper skips reservations whose payment is UNKNOWN

6. SOLID and pattern map

Principle or pattern	Where applied
SRP	Inventory, Payment, Order and Notification are separate services with one reason to change each
OCP	A new payment provider is a new adapter registered in <code>PaymentGatewayFactory</code>
LSP	Every adapter honours the <code>PaymentGateway</code> contract, including UNKNOWN on timeout
ISP	Small repository interfaces per aggregate
DIP	Services depend on <code>PaymentGateway</code> , <code>InventoryRepository</code> and the other

interfaces, not on concrete classes

Facade, saga	CheckoutFacade coordinates the purchase and compensates on failure
State	OrderState classes enforce valid order transitions
Adapter, Factory	Provider adapters, created by PaymentGatewayFactory
Decorator, circuit breaker	ResilientGateway wraps any gateway
Observer	Fulfilment and Notification subscribe to OrderConfirmed
Repository	SQL persistence is hidden behind interfaces